SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA0910 **COURSE NAME: Programming in Java**

COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 10 **Total Marks: 100**

ANNEXURE A **Coding Challenge**

Code of Conduct:

I B. Priyanka / 192421206 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B. Priyanka

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

```
import java.util.Scanner;

class Book {

    private int bookId;

    private String title;

    private String author;

    private double price;

    private boolean issued;

    Book(int bookId, String title, String author, double price) {

        this.bookId = bookId;

        this.title = title;

        this.author = author;

        this.price = price;

        this.issued = false;

    }

    public int getBookId() {

        return bookId;

    }

    public void issueBook() {

        if (!issued) {

            issued = true;

            System.out.println("Book issued successfully.");

        } else {

            System.out.println("Book is already issued.");
```

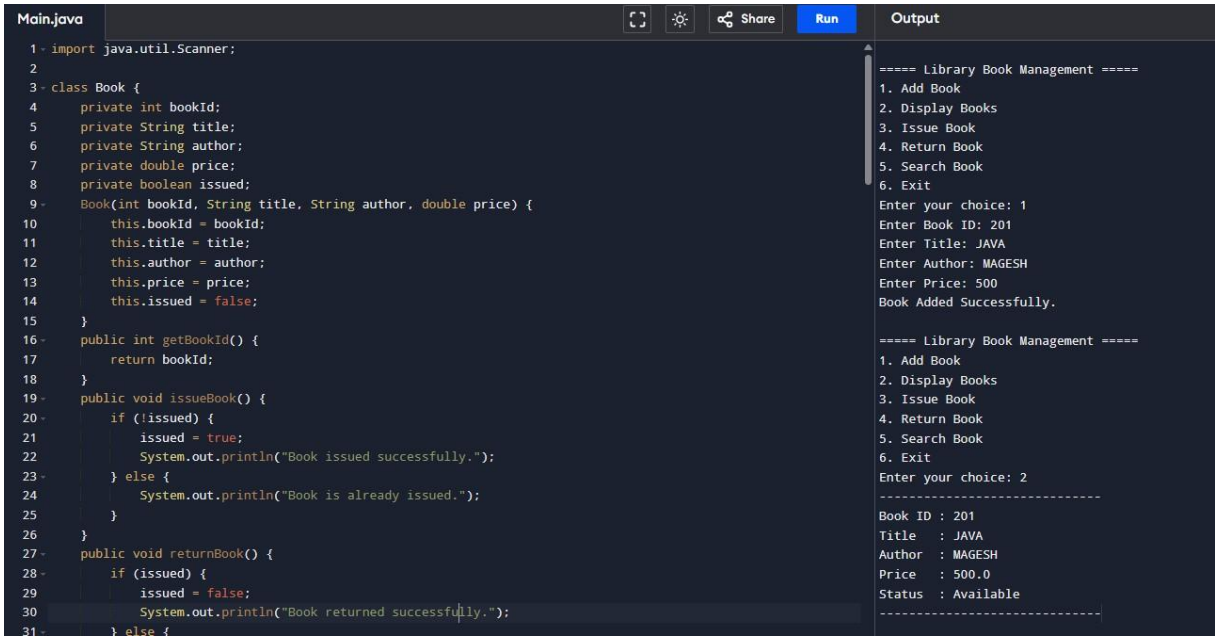
```
        }
    }
    public void returnBook() {
        if (issued) {
            issued = false;
            System.out.println("Book returned successfully.");
        } else {
            System.out.println("Book was not issued.");
        }
    }
    public void display() {
        System.out.println("----------------------");
        System.out.println("Book ID : " + bookId);
        System.out.println("Title   : " + title);
        System.out.println("Author  : " + author);
        System.out.println("Price   : " + price);
        System.out.println("Status  : " + (issued ? "Issued" : "Available"));
        System.out.println("----------------------");
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Book[] books = new Book[10];
        int count = 0;
        while (true) {
            System.out.println("\n===== Library Book Management =====");
            System.out.println("1. Add Book");
            System.out.println("2. Display Books");
```

```
System.out.println("3. Issue Book");
System.out.println("4. Return Book");
System.out.println("5. Search Book");
System.out.println("6. Exit");
System.out.print("Enter your choice: ");
int choice = sc.nextInt();
switch (choice) {
    case 1:
        if (count == books.length) {
            System.out.println("Library is Full!");
            break;
        }
        System.out.print("Enter Book ID: ");
        int id = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Title: ");
        String title = sc.nextLine();
        System.out.print("Enter Author: ");
        String author = sc.nextLine();
        System.out.print("Enter Price: ");
        double price = sc.nextDouble();
        books[count] = new Book(id, title, author, price);
        count++;
        System.out.println("Book Added Successfully.");
        break;
    case 2:
        if (count == 0) {
            System.out.println("No books available.");
        } else {
```

```
            for (int i = 0; i < count; i++) {
                books[i].display();
            }
        }
        break;
    case 3:
        System.out.print("Enter Book ID to Issue: ");
        id = sc.nextInt();
        boolean found = false;
        for (int i = 0; i < count; i++) {
            if (books[i].getBookId() == id) {
                books[i].issueBook();
                found = true;
                break;
            }
        }
        if (!found) {
            System.out.println("Book not found.");
        }
        break;
    case 4:
        System.out.print("Enter Book ID to Return: ");
        id = sc.nextInt();
        found = false;
        for (int i = 0; i < count; i++) {
            if (books[i].getBookId() == id) {
                books[i].returnBook();
                found = true;
                break;
```

```
            }
        }
        if (!found) {
            System.out.println("Book not found.");
        }
        break;
    case 5:
        System.out.print("Enter Book ID to Search: ");
        id = sc.nextInt();
        found = false;
        for (int i = 0; i < count; i++) {
            if (books[i].getBookId() == id) {
                books[i].display();
                found = true;
                break;
            }
        }
        if (!found) {
            System.out.println("Book not found.");
        }
        break;
    case 6:
        System.out.println("Thank You!");
        sc.close();
        System.exit(0);
    default:
        System.out.println("Invalid Choice.");
  }
}
```

```
    }
}
```



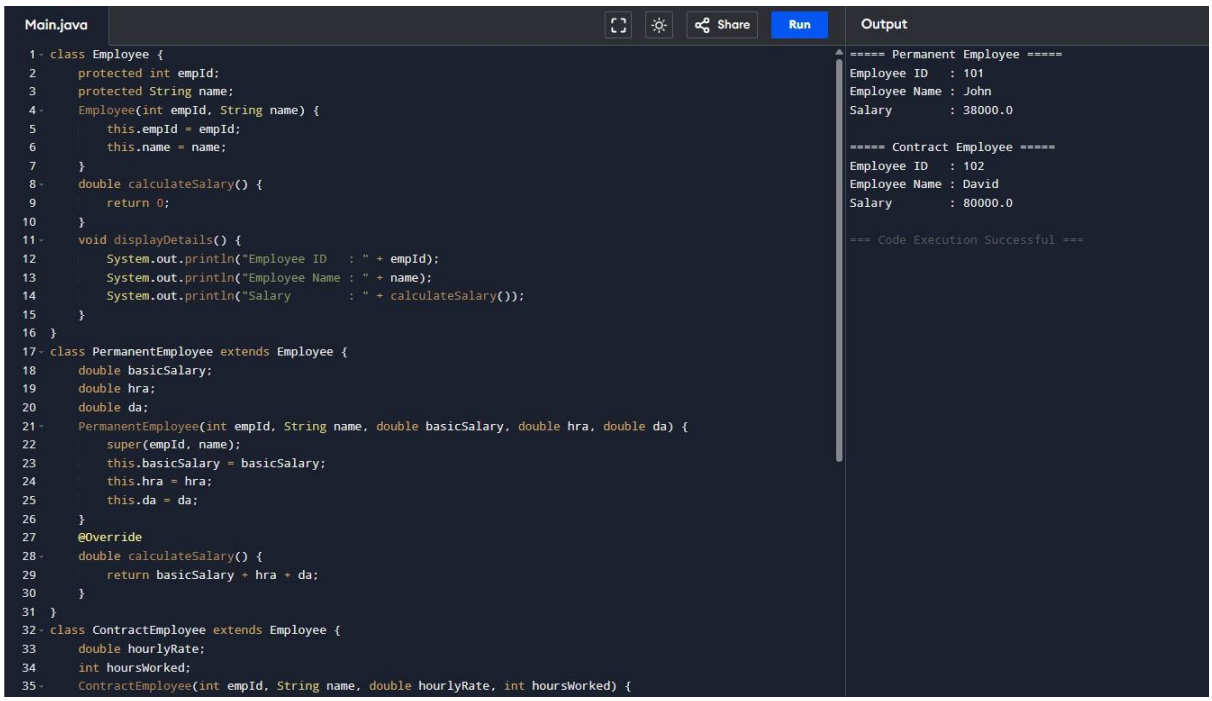
2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

```
class Employee {

    protected int empId;

    protected String name;

    Employee(int empId, String name) {

        this.empId = empId;

        this.name = name;

    }

    double calculateSalary() {

        return 0;

    }

    void displayDetails() {

        System.out.println("Employee ID  : " + empId);

        System.out.println("Employee Name : " + name);

        System.out.println("Salary       : " + calculateSalary());
```

```
    }
}
class PermanentEmployee extends Employee {
    double basicSalary;
    double hra;
    double da;
    PermanentEmployee(int empId, String name, double basicSalary, double hra, double da) {
        super(empId, name);
        this.basicSalary = basicSalary;
        this.hra = hra;
        this.da = da;
    }
    @Override
    double calculateSalary() {
        return basicSalary + hra + da;
    }
}
class ContractEmployee extends Employee {
    double hourlyRate;
    int hoursWorked;
    ContractEmployee(int empId, String name, double hourlyRate, int hoursWorked) {
        super(empId, name);
        this.hourlyRate = hourlyRate;
        this.hoursWorked = hoursWorked;
    }
    @Override
    double calculateSalary() {
        return hourlyRate * hoursWorked;
    }
```

```
}

public class Main {

    public static void main(String[] args) {

        Employee e1 = new PermanentEmployee(101, "John", 30000, 5000, 3000);

        Employee e2 = new ContractEmployee(102, "David", 500, 160);

        System.out.println("===== Permanent Employee =====");

        e1.displayDetails();

        System.out.println();

        System.out.println("===== Contract Employee =====");

        e2.displayDetails();

    }

}
```



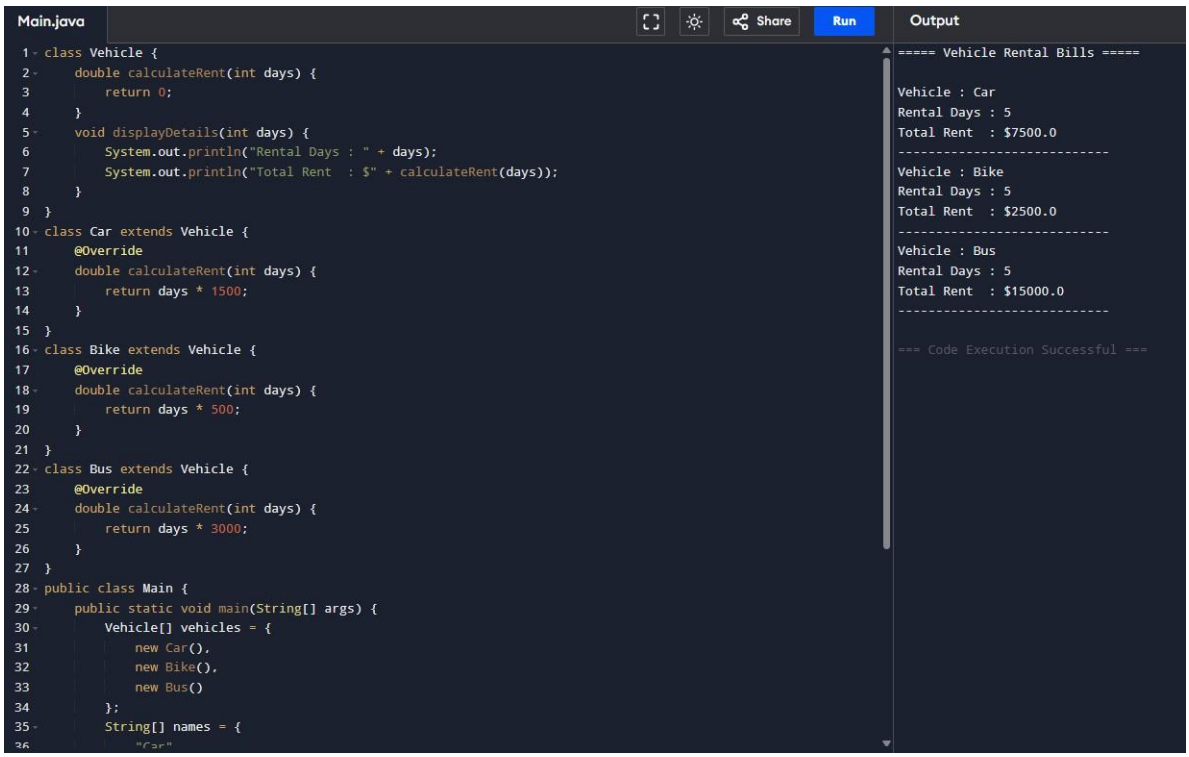
3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

```
class Vehicle {

    double calculateRent(int days) {

        return 0;
```

```
    }
    void displayDetails(int days) {
        System.out.println("Rental Days : " + days);
        System.out.println("Total Rent  : $" + calculateRent(days));
    }
}
class Car extends Vehicle {
    @Override
    double calculateRent(int days) {
        return days * 1500;
    }
}
class Bike extends Vehicle {
    @Override
    double calculateRent(int days) {
        return days * 500;
    }
}
class Bus extends Vehicle {
    @Override
    double calculateRent(int days) {
        return days * 3000;
    }
}
public class Main {
    public static void main(String[] args) {
        Vehicle[] vehicles = {
            new Car(),
            new Bike(),
```

```
            new Bus()
        };
        String[] names = {
            "Car",
            "Bike",
            "Bus"
        };
        int days = 5;
        System.out.println("===== Vehicle Rental Bills =====\n");
        for (int i = 0; i < vehicles.length; i++) {
            System.out.println("Vehicle : " + names[i]);
            vehicles[i].displayDetails(days);
            System.out.println("----------");
        }
    }
}
```



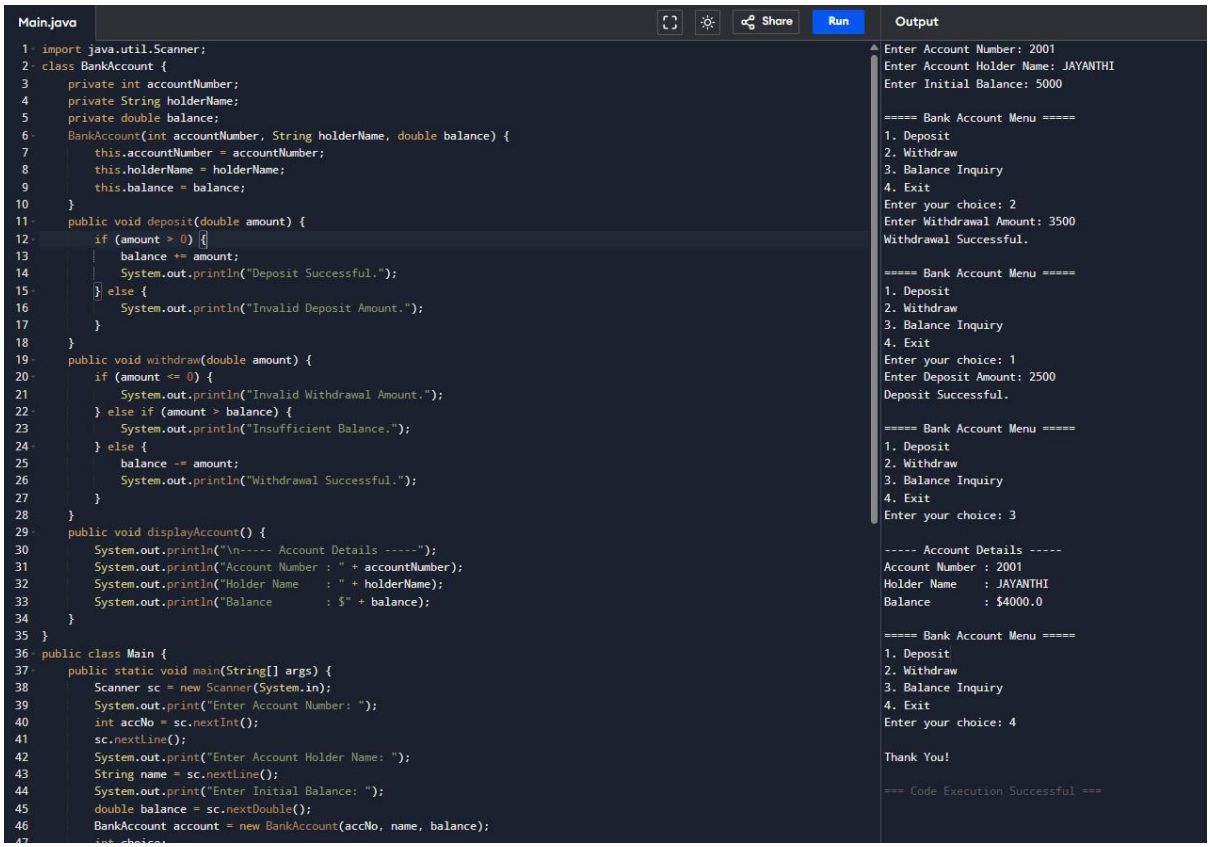
4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

```
import java.util.Scanner;

class BankAccount {

    private int accountNumber;

    private String holderName;

    private double balance;

    BankAccount(int accountNumber, String holderName, double balance) {

        this.accountNumber = accountNumber;

        this.holderName = holderName;

        this.balance = balance;

    }

    public void deposit(double amount) {

        if (amount > 0) {

            balance += amount;

            System.out.println("Deposit Successful.");

        } else {

            System.out.println("Invalid Deposit Amount.");

        }

    }

    public void withdraw(double amount) {

        if (amount <= 0) {

            System.out.println("Invalid Withdrawal Amount.");

        } else if (amount > balance) {

            System.out.println("Insufficient Balance.");

        } else {

            balance -= amount;
```

```
            System.out.println("Withdrawal Successful.");
        }
    }
    public void displayAccount() {
        System.out.println("\n----- Account Details ----- ");
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Holder Name    : " + holderName);
        System.out.println("Balance        : $" + balance);
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Account Number: ");
        int accNo = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Account Holder Name: ");
        String name = sc.nextLine();
        System.out.print("Enter Initial Balance: ");
        double balance = sc.nextDouble();
        BankAccount account = new BankAccount(accNo, name, balance);
        int choice;
        do {
            System.out.println("\n===== Bank Account Menu =====");
            System.out.println("1. Deposit");
            System.out.println("2. Withdraw");
            System.out.println("3. Balance Inquiry");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");
```

```
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.print("Enter Deposit Amount: ");
                    double deposit = sc.nextDouble();
                    account.deposit(deposit);
                    break;
                case 2:
                    System.out.print("Enter Withdrawal Amount: ");
                    double withdraw = sc.nextDouble();
                    account.withdraw(withdraw);
                    break;
                case 3:
                    account.displayAccount();
                    break;
                case 4:
                    System.out.println("Thank You!");
                    break;
                default:
                    System.out.println("Invalid Choice.");
            }
        } while (choice != 4);
        sc.close();
    }
}
```



5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

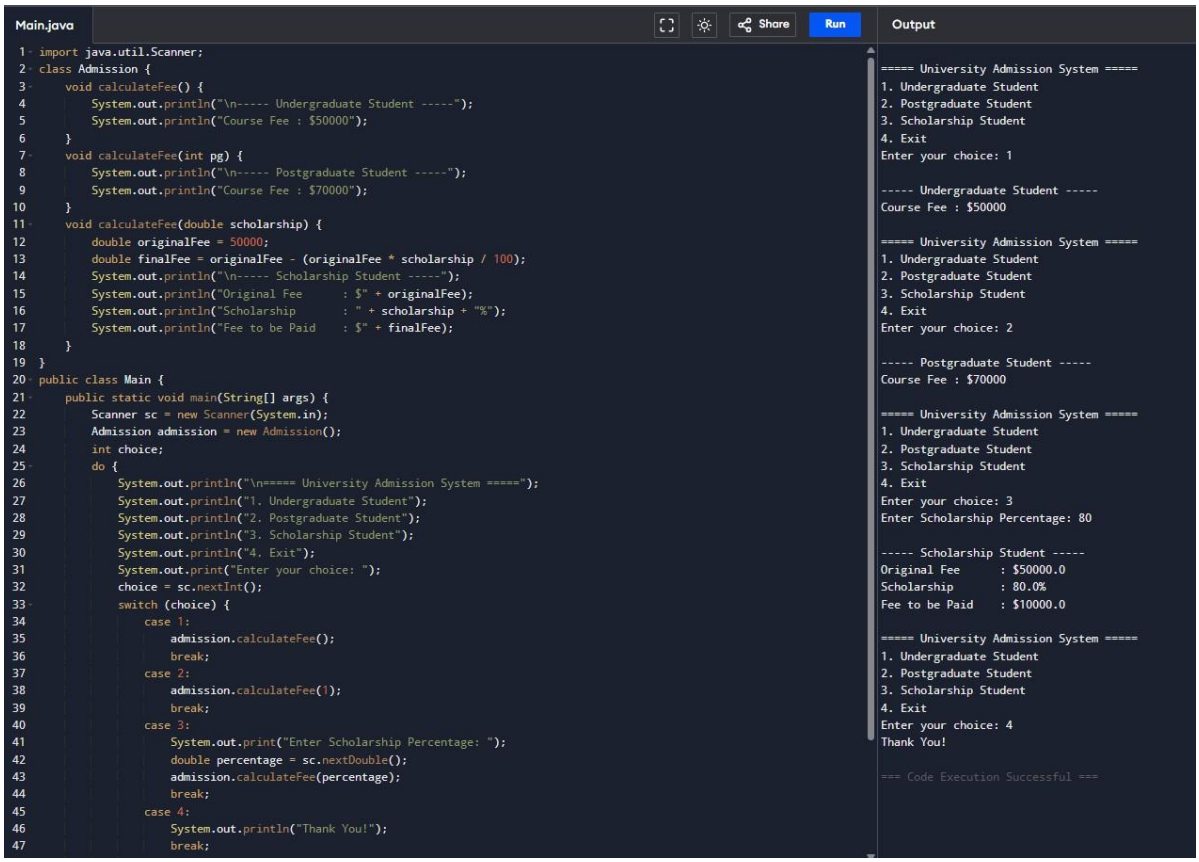
- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

```
import java.util.Scanner;

class Admission {

    void calculateFee() {

        System.out.println("\n----- Undergraduate Student----- ");

        System.out.println("Course Fee : $50000");

    }

    void calculateFee(int pg) {

        System.out.println("\n----- Postgraduate Student ----- ");

        System.out.println("Course Fee : $70000");

    }

    void calculateFee(double scholarship) {
```

```
        double originalFee = 50000;
        double finalFee = originalFee - (originalFee * scholarship / 100);
        System.out.println("\n----- Scholarship Student -----");
        System.out.println("Original Fee      : $" + originalFee);
        System.out.println("Scholarship       : " + scholarship + "%");
        System.out.println("Fee to be Paid    : $" + finalFee);
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Admission admission = new Admission();
        int choice;
        do {
            System.out.println("\n===== University Admission System =====");
            System.out.println("1. Undergraduate Student");
            System.out.println("2. Postgraduate Student");
            System.out.println("3. Scholarship Student");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    admission.calculateFee();
                    break;
                case 2:
                    admission.calculateFee(1);
                    break;
                case 3:
```

```
                System.out.print("Enter Scholarship Percentage: ");
                double percentage = sc.nextDouble();
                admission.calculateFee(percentage);
                break;
            case 4:
                System.out.println("Thank You!");
                break;
            default:
                System.out.println("Invalid Choice.");
        }
    } while (choice != 4);
    sc.close();
  }
}
```



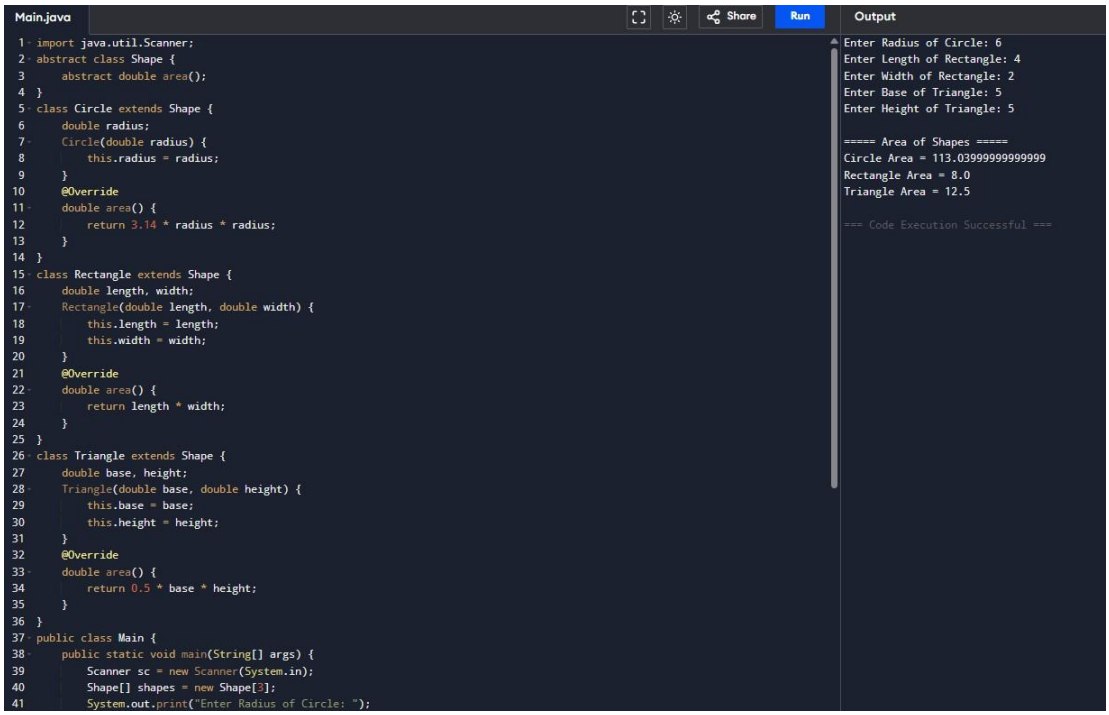
6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

```
import java.util.Scanner;

abstract class Shape {

    abstract double area();

}

class Circle extends Shape {

    double radius;

    Circle(double radius) {

        this.radius = radius;

    }

    @Override

    double area() {

        return 3.14 * radius * radius;

    }

}

class Rectangle extends Shape {

    double length, width;

    Rectangle(double length, double width) {

        this.length = length;

        this.width = width;

    }

    @Override

    double area() {

        return length * width;

    }

}
```

```
class Triangle extends Shape {
    double base, height;
    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }
    @Override
    double area() {
        return 0.5 * base * height;
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Shape[] shapes = new Shape[3];
        System.out.print("Enter Radius of Circle: ");
        double radius = sc.nextDouble();
        shapes[0] = new Circle(radius);
        System.out.print("Enter Length of Rectangle: ");
        double length = sc.nextDouble();
        System.out.print("Enter Width of Rectangle: ");
        double width = sc.nextDouble();
        shapes[1] = new Rectangle(length, width);
        System.out.print("Enter Base of Triangle: ");
        double base = sc.nextDouble();
        System.out.print("Enter Height of Triangle: ");
        double height = sc.nextDouble();
        shapes[2] = new Triangle(base, height);
        System.out.println("\n===== Area of Shapes =====");
```

```
        String[] shapeNames = {"Circle", "Rectangle", "Triangle"};
        for (int i = 0; i < shapes.length; i++) {
            System.out.println(shapeNames[i] + " Area = " + shapes[i].area());
        }
        sc.close();
    }
}
```



7. Hospital Management System using Interfaces

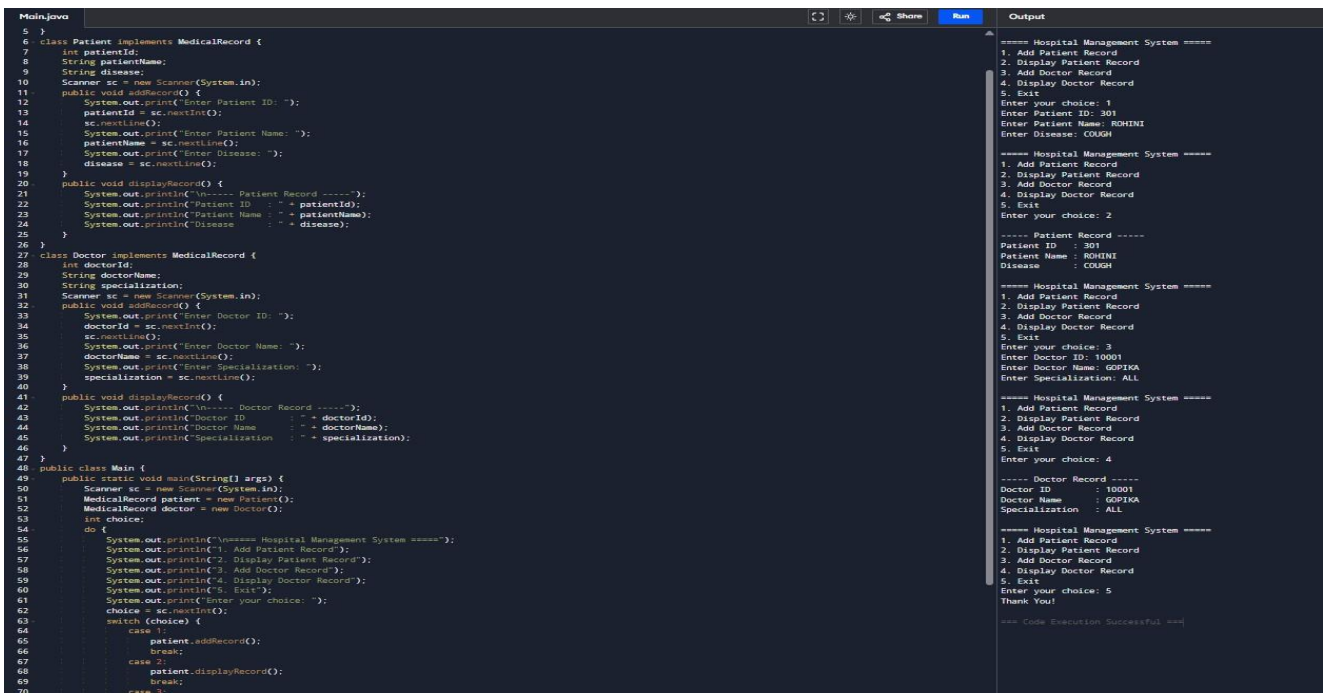
Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

```
import java.util.Scanner;

interface MedicalRecord {
    void addRecord();
    void displayRecord();
}

class Patient implements MedicalRecord {
    int patientId;
    String patientName;
```

```
    String disease;
    Scanner sc = new Scanner(System.in);
    public void addRecord() {
        System.out.print("Enter Patient ID: ");
        patientId = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Patient Name: ");
        patientName = sc.nextLine();
        System.out.print("Enter Disease: ");
        disease = sc.nextLine();
    }
    public void displayRecord() {
        System.out.println("\n----- Patient Record----- ");
        System.out.println("Patient ID   : " + patientId);
        System.out.println("Patient Name : " + patientName);
        System.out.println("Disease      : " + disease);
    }
}
class Doctor implements MedicalRecord {
    int doctorId;
    String doctorName;
    String specialization;
    Scanner sc = new Scanner(System.in);
    public void addRecord() {
        System.out.print("Enter Doctor ID: ");
        doctorId = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Doctor Name: ");
        doctorName = sc.nextLine();
```

```
        System.out.print("Enter Specialization: ");
        specialization = sc.nextLine();
    }
    public void displayRecord() {
        System.out.println("\n----- Doctor Record----- ");
        System.out.println("Doctor ID        : " + doctorId);
        System.out.println("Doctor Name      : " + doctorName);
        System.out.println("Specialization   : " + specialization);
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        MedicalRecord patient = new Patient();
        MedicalRecord doctor = new Doctor();
        int choice;
        do {
            System.out.println("\n===== Hospital Management System =====");
            System.out.println("1. Add Patient Record");
            System.out.println("2. Display Patient Record");
            System.out.println("3. Add Doctor Record");
            System.out.println("4. Display Doctor Record");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    patient.addRecord();
                    break;
```

```
            case 2:
                patient.displayRecord();
                break;
            case 3:
                doctor.addRecord();
                break;
            case 4:
                doctor.displayRecord();
                break;
            case 5:
                System.out.println("Thank You!");
                break;
            default:
                System.out.println("Invalid Choice.");
        }
    } while (choice != 5);
    sc.close();
  }
}
```



8. Online Shopping Order System using Packages

Create two packages:

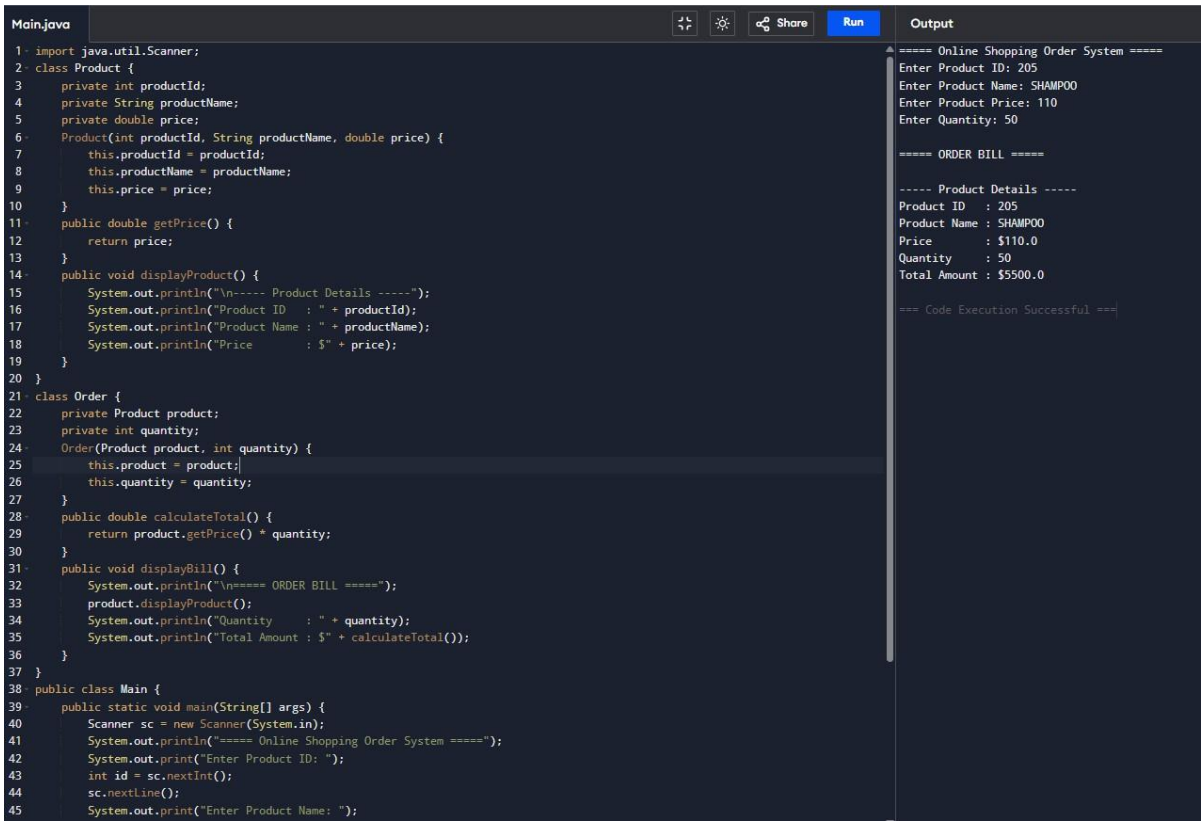
- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

```
import java.util.Scanner;

class Product {

    private int productId;

    private String productName;

    private double price;

    Product(int productId, String productName, double price) {

        this.productId = productId;

        this.productName = productName;

        this.price = price;

    }

    public double getPrice() {

        return price;

    }

    public void displayProduct() {

        System.out.println("\n----- Product Details ----- ");

        System.out.println("Product ID   : " + productId);

        System.out.println("Product Name : " + productName);

        System.out.println("Price        : $" + price);

    }

}

class Order {

    private Product product;

    private int quantity;
```

```
    Order(Product product, int quantity) {
        this.product = product;
        this.quantity = quantity;
    }
    public double calculateTotal() {
        return product.getPrice() * quantity;
    }
    public void displayBill() {
        System.out.println("\n===== ORDER BILL =====");
        product.displayProduct();
        System.out.println("Quantity      : " + quantity);
        System.out.println("Total Amount : $" + calculateTotal());
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("===== Online Shopping Order System =====");
        System.out.print("Enter Product ID: ");
        int id = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Product Name: ");
        String name = sc.nextLine();
        System.out.print("Enter Product Price: ");
        double price = sc.nextDouble();
        System.out.print("Enter Quantity: ");
        int quantity = sc.nextInt();
        Product p = new Product(id, name, price);
        Order o = new Order(p, quantity);
```

```
        o.displayBill();

        sc.close();

    }

}
```



9. Student Result Analyzer using Arrays of Objects

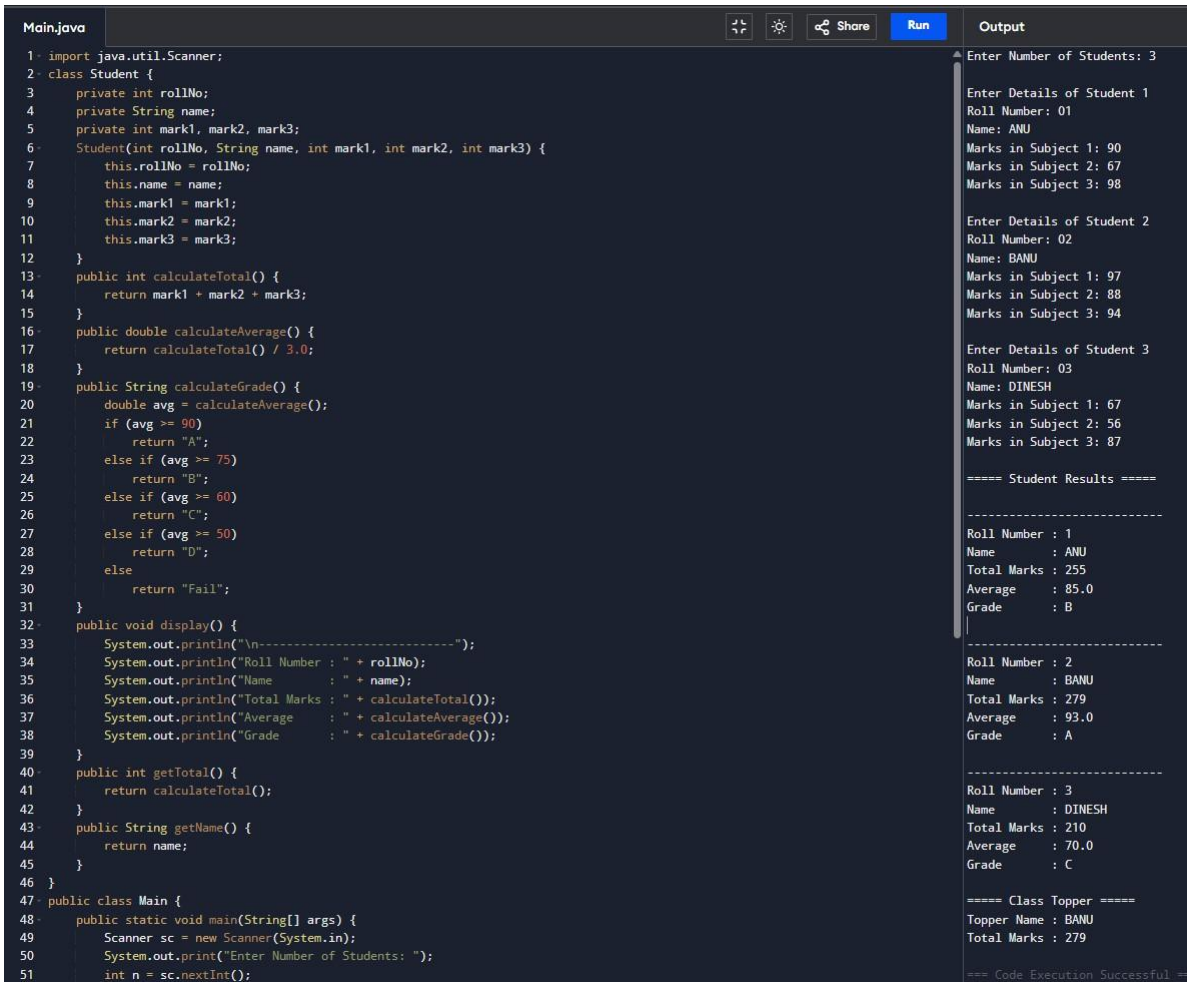
Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

```
import java.util.Scanner;

class Student {

    private int rollNo;

    private String name;

    private int mark1, mark2, mark3;

    Student(int rollNo, String name, int mark1, int mark2, int mark3) {

        this.rollNo = rollNo;

        this.name = name;

        this.mark1 = mark1;

        this.mark2 = mark2;
```

```
    this.mark3 = mark3;
}
public int calculateTotal() {
    return mark1 + mark2 + mark3;
}
public double calculateAverage() {
    return calculateTotal() / 3.0;
}
public String calculateGrade() {
    double avg = calculateAverage();
    if (avg >= 90)
        return "A";
    else if (avg >= 75)
        return "B";
    else if (avg >= 60)
        return "C";
    else if (avg >= 50)
        return "D";
    else
        return "Fail";
}
public void display() {
    System.out.println("\n---------------------------- ");
    System.out.println("Roll Number : " + rollNo);
    System.out.println("Name         : " + name);
    System.out.println("Total Marks : " + calculateTotal());
    System.out.println("Average      : " + calculateAverage());
    System.out.println("Grade        : " + calculateGrade());
}
```

```
    public int getTotal() {
        return calculateTotal();
    }
    public String getName() {
        return name;
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Number of Students: ");
        int n = sc.nextInt();
        Student[] students = new Student[n];
        for (int i = 0; i < n; i++) {
            System.out.println("\nEnter Details of Student " + (i + 1));
            System.out.print("Roll Number: ");
            int roll = sc.nextInt();
            sc.nextLine();
            System.out.print("Name: ");
            String name = sc.nextLine();
            System.out.print("Marks in Subject 1: ");
            int m1 = sc.nextInt();
            System.out.print("Marks in Subject 2: ");
            int m2 = sc.nextInt();
            System.out.print("Marks in Subject 3: ");
            int m3 = sc.nextInt();
            students[i] = new Student(roll, name, m1, m2, m3);
        }
        System.out.println("\n===== Student Results =====");
```

```
        Student topper = students[0];
        for (int i = 0; i < n; i++) {
            students[i].display();
            if (students[i].getTotal() > topper.getTotal()) {
                topper = students[i];
            }
        }
        System.out.println("\n===== Class Topper =====");
        System.out.println("Topper Name : " + topper.getName());
        System.out.println("Total Marks : " + topper.getTotal());
        sc.close();
    }
}
```



10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

```
import java.util.Scanner;
class Device {
    protected String deviceName;
    Device(String deviceName) {
        this.deviceName = deviceName;
    }
    void turnOn() {
        System.out.println(deviceName + " is ON");
    }
    void turnOff() {
        System.out.println(deviceName + " is OFF");
    }
    double powerConsumption() {
        return 0;
    }
    void displayPower() {
        System.out.println("Power Consumption : " + powerConsumption() + " Watts");
    }
}
class Light extends Device {
    Light() {
        super("Light");
    }

    @Override
    double powerConsumption() {
        return 10;
    }
}
class Fan extends Device {
```

```
    Fan() {
        super("Fan");
    }
    @Override
    double powerConsumption() {
        return 75;
    }
}
class AirConditioner extends Device {
    AirConditioner() {
        super("Air Conditioner");
    }
    @Override
    double powerConsumption() {
        return 1500;
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Device device;
        System.out.println("===== Smart Home Device Controller =====");
        System.out.println("1. Light");
        System.out.println("2. Fan");
        System.out.println("3. Air Conditioner");
        System.out.print("Enter your choice: ");
        int choice = sc.nextInt();
        switch (choice) {
            case 1:
                device = new Light();
                break;
            case 2:
                device = new Fan();
                break;
            case 3:
```

```
            device = new AirConditioner();
            break;
        default:
            System.out.println("Invalid Choice");
            sc.close();
            return;
        }
        System.out.println("\n===== Device Details =====");
        device.turnOn();
        device.displayPower();
        device.turnOff();
        sc.close();
    }
}
```